MySQL Interview Questions

Query-Based | Topic-Wise | With Solutions

Covers: SELECT · WHERE · Joins · Subqueries · Aggregates · DDL · DML · Constraints · Indexes · Views · Stored Procedures · Triggers · Window Functions · Normalization

100+ MySQL query-based interview questions across 15 topics. Every question comes with a written answer and working MySQL query.

SECTION 1 — SELECT & WHERE Clause

1 **Write a query to fetch all columns from the employees table.**

Ans: Use SELECT * to retrieve every column. In production, prefer listing columns explicitly for performance.

```
SELECT * FROM employees;
```

2 **Fetch only emp_name and salary for employees in department 3.**

Ans: Specify column names and use WHERE to filter rows by department.

```
SELECT emp_name, salary
FROM employees
WHERE dept_id = 3;
```

3 **Find employees with salary between 40000 and 80000.**

Ans: BETWEEN is inclusive on both ends — equivalent to salary >= 40000 AND salary <= 80000.

```
SELECT emp_name, salary
FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

4 **Retrieve employees whose name starts with 'A'.**

Ans: LIKE with % wildcard matches any number of characters after 'A'.

```
SELECT emp_name FROM employees
WHERE emp_name LIKE 'A%';
```

5 Fetch employees who belong to dept_id IN (1, 3, 5).

Ans: IN operator checks membership in a list — cleaner than multiple OR conditions.

```
SELECT emp_name, dept_id
FROM employees
WHERE dept_id IN (1, 3, 5);
```

6 Find employees where the manager_id is NULL.

Ans: NULL cannot be compared with = . Always use IS NULL or IS NOT NULL.

```
SELECT emp_name FROM employees
WHERE manager_id IS NULL;
```

■ *Tip: WHERE manager_id = NULL will always return 0 rows — that's a very common mistake.*

7 Retrieve employees NOT in department 2 and whose salary > 50000.

Ans: Combine != (or <>) with AND for multiple conditions.

```
SELECT emp_name, dept_id, salary
FROM employees
WHERE dept_id != 2
AND salary > 50000;
```

8 Fetch the top 5 highest-paid employees.

Ans: ORDER BY DESC sorts from highest to lowest; LIMIT restricts row count.

```
SELECT emp_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

SECTION 2 — Aggregate Functions, GROUP BY & HAVING

9 **Find the total number of employees in each department.**

Ans: GROUP BY groups rows by dept_id; COUNT(*) counts rows per group.

```
SELECT dept_id, COUNT(*) AS total_employees
FROM employees
GROUP BY dept_id;
```

10 **Find the maximum and minimum salary in the company.**

Ans: MAX() and MIN() are aggregate functions that work on the entire table without GROUP BY.

```
SELECT MAX(salary) AS highest_salary,
MIN(salary) AS lowest_salary
FROM employees;
```

11 **Get the average salary per department, only for departments with avg salary > 60000.**

Ans: Use HAVING to filter on aggregated values — WHERE cannot use aggregate functions.

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 60000;
```

12 **Count the number of employees in each department, sorted by count descending.**

Ans: Combine GROUP BY with ORDER BY to rank departments by headcount.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
ORDER BY emp_count DESC;
```

13 **Find departments where total salary bill exceeds 500000.**

Ans: SUM() totals salary per group; HAVING filters groups after aggregation.

```
SELECT dept_id, SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id
HAVING SUM(salary) > 500000;
```

14 What is the difference between COUNT(*) and COUNT(column)?

Ans: COUNT(*) counts all rows including NULLs. COUNT(column) counts only non-NULL values in that column.

```
-- Counts all rows
SELECT COUNT(*) FROM employees;

-- Counts only rows where manager_id is NOT NULL
SELECT COUNT(manager_id) FROM employees;
```

15 Find the second highest salary using aggregate functions.

Ans: MAX with a subquery excluding the maximum gives the second highest salary.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

■ *Tip: Alternatively use: SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;*

SECTION 3 — JOINs (INNER, LEFT, RIGHT, SELF, CROSS)

16 **Write a query to fetch employee names along with their department names.**

Ans: INNER JOIN returns rows where there is a match in both tables on the join condition.

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

17 **Fetch all employees even if they are not assigned to any department (LEFT JOIN).**

Ans: LEFT JOIN returns all rows from the left table and matched rows from the right. NULLs fill unmatched right columns.

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

18 **Fetch all departments even if no employee belongs to them (RIGHT JOIN).**

Ans: RIGHT JOIN returns all rows from the right table and matched rows from the left table.

```
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

19 **Write a SELF JOIN query to find each employee and their manager's name.**

Ans: A self join aliases the same table twice — one alias for employees, another for managers.

```
SELECT e.emp_name AS employee,
m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

20 **Write a CROSS JOIN query to generate all combinations of colors and sizes.**

Ans: CROSS JOIN produces a Cartesian product — every row of table A paired with every row of table B.

```
SELECT c.color_name, s.size_label
FROM colors c
CROSS JOIN sizes s;
```

21 **Find employees who share the same department using a self join.**

Ans: Join the same table on dept_id but exclude the same employee pairing with themselves.

```
SELECT a.emp_name AS emp1, b.emp_name AS emp2, a.dept_id
FROM employees a
JOIN employees b ON a.dept_id = b.dept_id
WHERE a.emp_id < b.emp_id;
```

22 **What is the difference between INNER JOIN and LEFT JOIN?**

Ans: INNER JOIN: only rows with matches in BOTH tables are returned. LEFT JOIN: ALL rows from the left table are returned; NULLs appear for the right table where there is no match.

```
-- INNER JOIN: excludes employees with no department
SELECT e.emp_name, d.dept_name FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN: includes employees even without a department
SELECT e.emp_name, d.dept_name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

SECTION 4 — Subqueries

23 **Find employees who earn more than the average salary.**

Ans: The subquery calculates the overall average; the outer query filters rows above it.

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

24 **Find the employee with the highest salary using a subquery.**

Ans: The inner query returns MAX(salary); the outer query finds the employee matching that value.

```
SELECT emp_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

25 **Fetch employees who work in departments that have at least one project.**

Ans: Multi-row subquery with IN — inner query returns a set of matching dept_ids.

```
SELECT emp_name, dept_id
FROM employees
WHERE dept_id IN (
SELECT DISTINCT dept_id FROM projects
);
```

26 **Find employees earning more than ALL employees in department 5.**

Ans: ALL operator compares a value against every value in the subquery result set.

```
SELECT emp_name, salary
FROM employees
WHERE salary > ALL (
SELECT salary FROM employees WHERE dept_id = 5
);
```

27 **Write a correlated subquery to find employees earning more than their department average.**

Ans: A correlated subquery references the outer query's row — it re-executes for every row.

```
SELECT e1.emp_name, e1.salary, e1.dept_id
FROM employees e1
WHERE e1.salary > (
SELECT AVG(e2.salary)
FROM employees e2
WHERE e2.dept_id = e1.dept_id
);
```

28 **Use a subquery in the FROM clause (Derived Table) to get department averages.**

Ans: A subquery in FROM creates a derived/inline table. It must be given an alias.

```
SELECT dept_id, avg_sal
FROM (
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
) AS dept_avg
WHERE avg_sal > 55000;
```

SECTION 5 — DDL (CREATE, ALTER, DROP, TRUNCATE, RENAME)

29 **Create a table called employees with appropriate columns and constraints.**

Ans: Use CREATE TABLE with data types, NOT NULL, DEFAULT, PRIMARY KEY, and FOREIGN KEY constraints.

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY AUTO_INCREMENT,
emp_name VARCHAR(100) NOT NULL,
email VARCHAR(150) UNIQUE,
salary DECIMAL(10,2) DEFAULT 30000.00,
dept_id INT,
hire_date DATE DEFAULT (CURRENT_DATE),
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

30 **Add a new column phone_number to the employees table.**

Ans: ALTER TABLE with ADD COLUMN modifies an existing table structure.

```
ALTER TABLE employees
ADD COLUMN phone_number VARCHAR(15) AFTER emp_name;
```

31 **Change the data type of salary column from DECIMAL(10,2) to DECIMAL(12,2).**

Ans: ALTER TABLE with MODIFY COLUMN changes a column's definition.

```
ALTER TABLE employees
MODIFY COLUMN salary DECIMAL(12, 2) NOT NULL;
```

32 **Rename the column emp_name to employee_name.**

Ans: In MySQL 8+, use RENAME COLUMN. In older versions, use CHANGE.

```
-- MySQL 8+
ALTER TABLE employees RENAME COLUMN emp_name TO employee_name;

-- MySQL 5.7 and below
ALTER TABLE employees CHANGE emp_name employee_name VARCHAR(100);
```

33 What is the difference between TRUNCATE and DELETE?

Ans: DELETE is DML and can be rolled back; fires row triggers; supports WHERE; slow on large tables. TRUNCATE is DDL; removes all rows instantly; cannot be rolled back; resets AUTO_INCREMENT; much faster.

```
-- DELETE: removes specific rows, can rollback
DELETE FROM logs WHERE log_date < '2023-01-01';

-- TRUNCATE: wipes entire table instantly
TRUNCATE TABLE logs;
```

34 Drop a column from a table and then drop the entire table.

Ans: ALTER TABLE with DROP COLUMN removes a column. DROP TABLE removes the entire structure.

```
-- Drop a column
ALTER TABLE employees DROP COLUMN phone_number;

-- Drop the entire table
DROP TABLE IF EXISTS temp_employees;
```

SECTION 6 — DML (INSERT, UPDATE, DELETE)

35 **Insert a single record and multiple records into employees table.**

Ans: Single INSERT provides one VALUES tuple. Bulk INSERT lists multiple tuples separated by commas — much faster than multiple single inserts.

```
-- Single insert
INSERT INTO employees (emp_name, salary, dept_id)
VALUES ('Ravi Kumar', 65000, 2);

-- Bulk insert
INSERT INTO employees (emp_name, salary, dept_id) VALUES
('Anita Singh', 72000, 1),
('Kiran Patel', 58000, 3),
('Suresh Rao', 81000, 2);
```

36 **Update the salary of all employees in department 3 by giving a 15% raise.**

Ans: UPDATE without WHERE affects all rows — always double-check your WHERE condition.

```
UPDATE employees
SET salary = salary * 1.15
WHERE dept_id = 3;
```

37 **Delete all employees who have not been assigned to any department.**

Ans: Use IS NULL to find employees with no dept_id assigned.

```
DELETE FROM employees
WHERE dept_id IS NULL;
```

38 **Write an INSERT...SELECT to copy data from one table to another.**

Ans: INSERT...SELECT inserts the result of a SELECT query directly — useful for data migration and archiving.

```
-- Archive inactive employees
INSERT INTO archived_employees (emp_id, emp_name, salary, dept_id)
SELECT emp_id, emp_name, salary, dept_id
FROM employees
WHERE status = 'inactive';
```

39 **Update salary of multiple employees based on their department using CASE.**

Ans: CASE inside SET allows conditional updates — avoids multiple UPDATE statements.

```
UPDATE employees
SET salary = CASE dept_id
WHEN 1 THEN salary * 1.20
WHEN 2 THEN salary * 1.15
WHEN 3 THEN salary * 1.10
ELSE salary
END;
```

SECTION 7 — Constraints (PK, FK, UNIQUE, CHECK, DEFAULT)

40 What is the difference between PRIMARY KEY and UNIQUE KEY?

Ans: A table has only ONE PRIMARY KEY and it cannot be NULL. A table can have MULTIPLE UNIQUE keys and they can contain one NULL. Both enforce uniqueness; PRIMARY KEY also serves as the main row identifier.

```
CREATE TABLE users (
user_id INT PRIMARY KEY AUTO_INCREMENT, -- one PK, no NULLs
email VARCHAR(150) UNIQUE, -- multiple UKs allowed
username VARCHAR(50) UNIQUE
);
```

41 Add a CHECK constraint to ensure age is between 18 and 60.

Ans: CHECK constraints validate column values against a condition at insert/update time.

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
age INT CHECK (age BETWEEN 18 AND 60),
salary DECIMAL(10,2) CHECK (salary > 0)
);
-- Add CHECK to existing table
ALTER TABLE employees ADD CONSTRAINT chk_age CHECK (age >= 18);
```

42 Create a table with a composite primary key.

Ans: A composite primary key uses two or more columns together to uniquely identify a row.

```
CREATE TABLE order_items (
order_id INT,
product_id INT,
quantity INT NOT NULL,
PRIMARY KEY (order_id, product_id) -- Composite PK
);
```

43 **Add a FOREIGN KEY with ON DELETE CASCADE to employees table.**

Ans: ON DELETE CASCADE automatically deletes child rows when the parent row is deleted.

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
dept_id INT,
FOREIGN KEY (dept_id)
REFERENCES departments(dept_id)
ON DELETE CASCADE
ON UPDATE CASCADE
);
```

SECTION 8 — String Functions & Date Functions

44 **Write queries using UPPER, LOWER, LENGTH, SUBSTR, and CONCAT.**

Ans: MySQL string functions manipulate text data directly in queries without needing external code.

```
SELECT UPPER('hello mysql') AS upper_case;
SELECT LOWER('HELLO MYSQL') AS lower_case;
SELECT LENGTH('Database') AS char_count;
SELECT SUBSTR('Database', 1, 4) AS first_4;
SELECT CONCAT(emp_name,' - ',dept_id) AS label FROM employees;
```

45 **Find employees whose names contain the letter 'a' at position 2.**

Ans: Underscore _ in LIKE matches exactly one character — useful for positional matching.

```
SELECT emp_name FROM employees
WHERE emp_name LIKE '_a%';
```

46 **Write queries using REPLACE, TRIM, REVERSE, and INSTR.**

Ans: These functions clean, transform, and search within string values.

```
SELECT REPLACE('Hello World', 'World', 'MySQL'); -- Hello MySQL
SELECT TRIM(' SQL '); -- 'SQL'
SELECT LTRIM(' SQL'); -- 'SQL'
SELECT RTRIM('SQL '); -- 'SQL'
SELECT REVERSE('MySQL'); -- LQSyM
SELECT INSTR('Hello World', 'World'); -- 7
```

47 **Write queries to get current date, format a date, and add/subtract days.**

Ans: MySQL provides built-in date functions for common date arithmetic and formatting.

```
SELECT CURDATE(); -- 2024-06-15
SELECT NOW(); -- 2024-06-15 10:30:00
SELECT DATE_FORMAT(NOW(), '%d-%m-%Y'); -- 15-06-2024
SELECT DATE_ADD('2024-01-01', INTERVAL 30 DAY); -- 2024-01-31
SELECT DATE_SUB(CURDATE(), INTERVAL 1 MONTH);
SELECT DATEDIFF('2024-12-31', '2024-01-01') AS days; -- 365
SELECT YEAR(hire_date), MONTH(hire_date) FROM employees;
```

48 **Find all employees hired in the year 2023.**

Ans: YEAR() extracts the year part from a date column.

```
SELECT emp_name, hire_date
FROM employees
WHERE YEAR(hire_date) = 2023;
```

SECTION 9 — Numeric Functions

49 **Write queries using ROUND, FLOOR, CEIL, ABS, MOD, POWER, SQRT.**

Ans: Numeric functions perform mathematical operations on column values or literals.

```
SELECT ROUND(3567.789, 2) AS rounded; -- 3567.79
SELECT FLOOR(4.9) AS floor_val; -- 4
SELECT CEIL(4.1) AS ceil_val; -- 5
SELECT ABS(-350) AS absolute; -- 350
SELECT MOD(17, 5) AS remainder; -- 2
SELECT POWER(2, 8) AS power_val; -- 256
SELECT SQRT(144) AS square_rt; -- 12
```

50 **Calculate each employee's salary after a 12% tax deduction, rounded to 2 decimals.**

Ans: Combine arithmetic expressions with ROUND for formatted financial calculations.

```
SELECT emp_name,
salary,
ROUND(salary * 0.88, 2) AS after_tax_salary
FROM employees;
```

SECTION 10 — Indexes & Views

51 Create a simple index, a unique index, and a composite index.

Ans: Indexes speed up SELECT queries but add overhead to INSERT/UPDATE/DELETE. Choose columns used frequently in WHERE, JOIN, and ORDER BY.

```
-- Simple index
CREATE INDEX idx_emp_name ON employees(emp_name);

-- Unique index (also enforces uniqueness)
CREATE UNIQUE INDEX idx_email ON employees(email);

-- Composite index (useful for multi-column WHERE)
CREATE INDEX idx_dept_salary ON employees(dept_id, salary);

-- View and drop indexes
SHOW INDEX FROM employees;
DROP INDEX idx_emp_name ON employees;
```

52 What is the difference between a clustered and non-clustered index?

Ans: A clustered index physically reorders table data — only ONE per table (PRIMARY KEY by default). A non-clustered index is a separate lookup structure pointing to rows — a table can have many.

```
-- Primary key = clustered index (auto-created)
CREATE TABLE employees (emp_id INT PRIMARY KEY, ...);

-- Non-clustered index (separate structure)
CREATE INDEX idx_dept ON employees(dept_id);
```

53 Create a view to show employee name, department name, and salary.

Ans: A view is a stored SELECT query — a virtual table. It doesn't store data but simplifies complex queries.

```
CREATE VIEW emp_dept_view AS
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;

-- Query the view
SELECT * FROM emp_dept_view WHERE salary > 60000;

-- Drop the view
DROP VIEW IF EXISTS emp_dept_view;
```

54 **Create a view that shows only high-earning employees (salary > 75000).**

Ans: Views can be used to restrict row-level access — users can query the view without seeing all rows.

```
CREATE VIEW high_earners AS
SELECT emp_id, emp_name, salary, dept_id
FROM employees
WHERE salary > 75000;

SELECT * FROM high_earners ORDER BY salary DESC;
```

SECTION 11 — Window Functions (RANK, DENSE_RANK, ROW_NUMBER)

55 **Rank employees by salary within each department using RANK().**

Ans: RANK() assigns same rank to ties but skips the next rank (e.g., 1,1,3). PARTITION BY resets ranking per department.

```
SELECT emp_name, dept_id, salary,
RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;
```

56 **What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?**

Ans: ROW_NUMBER: always unique (1,2,3,4). RANK: ties share rank, gaps follow (1,1,3,4). DENSE_RANK: ties share rank, no gaps (1,1,2,3).

```
SELECT emp_name, salary,
ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
RANK() OVER (ORDER BY salary DESC) AS rnk,
DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

57 **Find the top-1 highest-paid employee per department using ROW_NUMBER().**

Ans: Wrap the window function in a subquery and filter WHERE row_num = 1 to get the top record per partition.

```
SELECT emp_name, dept_id, salary
FROM (
SELECT emp_name, dept_id, salary,
ROW_NUMBER() OVER (
PARTITION BY dept_id ORDER BY salary DESC
) AS rn
FROM employees
) ranked
WHERE rn = 1;
```

58 **Calculate a running total of salaries ordered by hire_date.**

Ans: SUM() OVER with ORDER BY creates a cumulative running total — rows accumulate as they appear.

```
SELECT emp_name, hire_date, salary,
SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM employees;
```

SECTION 12 — Stored Procedures

59 **Write a basic stored procedure to fetch all employees.**

Ans: Stored procedures are named, precompiled SQL blocks called with CALL. DELIMITER changes the statement terminator so semicolons inside the body don't end the CREATE statement prematurely.

```
DELIMITER //
CREATE PROCEDURE GetAllEmployees()
BEGIN
SELECT emp_id, emp_name, salary, dept_id
FROM employees
ORDER BY emp_id;
END //
DELIMITER ;

-- Call the procedure
CALL GetAllEmployees();
```

60 **Write a procedure with an IN parameter to get employees by department.**

Ans: IN parameters accept values from the caller. The procedure uses the parameter in its logic.

```
DELIMITER //
CREATE PROCEDURE GetEmpByDept(IN p_dept_id INT)
BEGIN
SELECT emp_name, salary
FROM employees
WHERE dept_id = p_dept_id;
END //
DELIMITER ;

CALL GetEmpByDept(3);
```

61 **Write a procedure with an OUT parameter to return total employee count.**

Ans: OUT parameters send a value back to the caller. Use SELECT ... INTO to assign a value to the OUT parameter.

```
DELIMITER //
CREATE PROCEDURE GetEmpCount(OUT p_count INT)
BEGIN
SELECT COUNT(*) INTO p_count
FROM employees;
END //
DELIMITER ;

CALL GetEmpCount(@total);
SELECT @total AS total_employees;
```

62 **Write a procedure with IF-ELSE logic to classify salary grade.**

Ans: Conditional logic inside procedures allows business rules to run server-side.

```
DELIMITER //
CREATE PROCEDURE GetSalaryGrade(IN p_sal DECIMAL(10,2), OUT p_grade VARCHAR(5))
BEGIN
IF p_sal >= 80000 THEN SET p_grade = 'A';
ELSEIF p_sal >= 60000 THEN SET p_grade = 'B';
ELSEIF p_sal >= 40000 THEN SET p_grade = 'C';
ELSE SET p_grade = 'D';
END IF;
END //
DELIMITER ;

CALL GetSalaryGrade(72000, @grade);
SELECT @grade;
```

63 **Write a procedure with a transaction and error handler.**

Ans: DECLARE EXIT HANDLER FOR SQLEXCEPTION catches errors and rolls back the transaction, ensuring atomicity.

```
DELIMITER //
CREATE PROCEDURE TransferSalary(
IN p_from INT, IN p_to INT, IN p_amount DECIMAL(10,2))
BEGIN
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
ROLLBACK;
SELECT 'Transaction failed - rolled back' AS result;
END;
START TRANSACTION;
UPDATE employees SET salary = salary - p_amount WHERE emp_id = p_from;
UPDATE employees SET salary = salary + p_amount WHERE emp_id = p_to;
COMMIT;
SELECT 'Transfer successful' AS result;
END //
DELIMITER ;
```

SECTION 13 — Triggers (BEFORE / AFTER, INSERT / UPDATE / DELETE)

64 **Write an AFTER INSERT trigger to log new employee additions.**

Ans: AFTER INSERT triggers fire after a successful INSERT. NEW refers to the newly inserted row's values.

```
DELIMITER //
CREATE TRIGGER trg_after_emp_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
INSERT INTO audit_log(action, emp_id, action_time, done_by)
VALUES ('INSERT', NEW.emp_id, NOW(), USER());
END //
DELIMITER ;
```

65 **Write a BEFORE UPDATE trigger to prevent salary from being set below 10000.**

Ans: BEFORE triggers can validate or modify data before it is committed. SIGNAL raises a custom error.

```
DELIMITER //
CREATE TRIGGER trg_before_salary_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < 10000 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary cannot be below 10000';
END IF;
END //
DELIMITER ;
```

66 **Write an AFTER DELETE trigger to archive deleted employee records.**

Ans: AFTER DELETE triggers use OLD to access the deleted row's values before they are permanently removed.

```
DELIMITER //
CREATE TRIGGER trg_after_emp_delete
AFTER DELETE ON employees
FOR EACH ROW
BEGIN
INSERT INTO deleted_employees(
emp_id, emp_name, salary, dept_id, deleted_at)
VALUES(
OLD.emp_id, OLD.emp_name, OLD.salary, OLD.dept_id, NOW());
END //
DELIMITER ;
```

67 **What is the difference between BEFORE and AFTER triggers? When to use each?**

Ans: BEFORE triggers fire before the DML operation — use them to validate or modify data before commit. AFTER triggers fire after the operation succeeds — use them for auditing, logging, or cascading changes.

```
-- BEFORE: validate before saving
CREATE TRIGGER trg_chk_age BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
IF NEW.age < 18 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Age must be >= 18';
END IF;
END;
```

SECTION 14 — Transactions (TCL: COMMIT, ROLLBACK, SAVEPOINT)

68 **Write a transaction to transfer money between two accounts safely.**

Ans: A transaction wraps multiple statements into an atomic unit. If any step fails, ROLLBACK undoes all changes.

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 101;
UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 202;

-- If both succeed:
COMMIT;

-- If any error occurs:
-- ROLLBACK;
```

69 **Demonstrate SAVEPOINT and ROLLBACK TO SAVEPOINT.**

Ans: SAVEPOINT creates a named checkpoint inside a transaction. You can roll back to it without undoing the entire transaction.

```
START TRANSACTION;

INSERT INTO orders (order_id, customer_id) VALUES (1001, 5);
SAVEPOINT after_order;

INSERT INTO order_items VALUES (1001, 201, 3);
SAVEPOINT after_items;

-- Oops, wrong payment amount — roll back only the payment
INSERT INTO payments VALUES (1001, 9999.00);
ROLLBACK TO SAVEPOINT after_items;

-- Redo the payment correctly
INSERT INTO payments VALUES (1001, 450.00);
COMMIT;
```

70 What is the difference between COMMIT and ROLLBACK?

Ans: COMMIT permanently saves all changes made in the current transaction to the database. ROLLBACK undoes all changes made since the transaction started (or to a SAVEPOINT), restoring the previous state.

```
-- Successful case
START TRANSACTION;
UPDATE employees SET salary = 90000 WHERE emp_id = 5;
COMMIT; -- Change is permanent

-- Failed case
START TRANSACTION;
DELETE FROM employees WHERE dept_id = 2; -- Oops!
ROLLBACK; -- All changes undone
```

SECTION 15 — Normalization (1NF, 2NF, 3NF) — Schema Queries

71 **Show a table that violates 1NF and write the corrected schema.**

Ans: 1NF requires atomic values — no multi-valued or repeating group columns. Fix by splitting values into separate rows.

```
-- VIOLATES 1NF: multiple skills in one cell
-- | emp_id | emp_name | skills |
-- | 1 | Alice | Java, Python, SQL |

-- CORRECTED to 1NF:
CREATE TABLE emp_skills (
emp_id INT,
skill VARCHAR(50),
PRIMARY KEY (emp_id, skill)
);
INSERT INTO emp_skills VALUES (1,'Java'),(1,'Python'),(1,'SQL');
```

72 **Demonstrate a 2NF violation and the corrected schema.**

Ans: 2NF: no partial dependency — every non-key column must depend on the FULL composite primary key, not just part of it.

```
-- VIOLATES 2NF: student_name depends only on student_id,
-- not on the composite key (student_id, course_id)
-- | student_id | course_id | student_name | grade |

-- CORRECTED to 2NF: split into two tables
CREATE TABLE students (
student_id INT PRIMARY KEY,
student_name VARCHAR(100)
);
CREATE TABLE enrollments (
student_id INT,
course_id INT,
grade CHAR(2),
PRIMARY KEY (student_id, course_id)
);
```

73 **Demonstrate a 3NF violation and the corrected schema.**

Ans: 3NF: no transitive dependency — non-key columns must depend ONLY on the primary key, not on other non-key columns.

```
-- VIOLATES 3NF: dept_name depends on dept_id (not emp_id)
-- | emp_id | emp_name | dept_id | dept_name |

-- CORRECTED to 3NF:
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
emp_name VARCHAR(100),
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

SECTION 16 — Tricky & Most-Asked Query Problems

74 **Find duplicate records in the employees table based on emp_name.**

Ans: GROUP BY the duplicate column and use HAVING COUNT(*) > 1 to filter groups with more than one occurrence.

```
SELECT emp_name, COUNT(*) AS occurrences
FROM employees
GROUP BY emp_name
HAVING COUNT(*) > 1;
```

75 **Delete duplicate rows keeping only the one with the lowest emp_id.**

Ans: Use NOT IN with a subquery that selects the MIN(emp_id) per group — keeps one row per duplicate set.

```
DELETE FROM employees
WHERE emp_id NOT IN (
SELECT MIN(emp_id)
FROM employees
GROUP BY emp_name, dept_id
);
```

76 **Find the Nth highest salary without using LIMIT/OFFSET.**

Ans: Correlated subquery counts how many employees earn more — when that count equals N-1, that row has the Nth highest.

```
-- 3rd highest salary (set N = 3)
SELECT DISTINCT salary
FROM employees e1
WHERE 2 = (
SELECT COUNT(DISTINCT salary)
FROM employees e2
WHERE e2.salary > e1.salary
);
```

■ *Tip: Replace the '2' with N-1 to get the Nth highest salary.*

77 **Swap the values of two columns (e.g., swap first_name and last_name) in a single UPDATE.**

Ans: MySQL allows reading old values during an UPDATE — swap using a single SET with both assignments.

```
UPDATE employees
SET first_name = last_name,
last_name = first_name
WHERE emp_id = 101;
```

78 **Find employees who joined in the last 30 days.**

Ans: DATE_SUB subtracts an interval from a date — CURDATE() minus 30 days gives the cutoff date.

```
SELECT emp_name, hire_date
FROM employees
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

79 **Find all employees who do NOT have a manager (top-level employees).**

Ans: manager_id IS NULL identifies employees at the top of the hierarchy with no parent.

```
SELECT emp_id, emp_name
FROM employees
WHERE manager_id IS NULL;
```

80 **Display employees whose salary is above average but in a department with below-average headcount.**

Ans: Combine two subqueries — one for salary threshold, one for departments with low headcount.

```
SELECT e.emp_name, e.salary, e.dept_id
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees)
AND e.dept_id IN (
SELECT dept_id FROM employees
GROUP BY dept_id
HAVING COUNT(*) < (SELECT AVG(cnt) FROM (
SELECT COUNT(*) AS cnt FROM employees GROUP BY dept_id
) t)
);
```

81 **Write a query to pivot department-wise salary totals into columns.**

Ans: Use conditional aggregation with SUM + CASE to create column-per-department totals in a single row.

```
SELECT
SUM(CASE WHEN dept_id = 1 THEN salary ELSE 0 END) AS dept1_total,
SUM(CASE WHEN dept_id = 2 THEN salary ELSE 0 END) AS dept2_total,
SUM(CASE WHEN dept_id = 3 THEN salary ELSE 0 END) AS dept3_total
FROM employees;
```

82 **Find pairs of employees in the same department earning the same salary.**

Ans: Self-join on dept_id AND salary, with emp_id less-than to avoid duplicate pairs.

```
SELECT a.emp_name, b.emp_name, a.salary, a.dept_id
FROM employees a
JOIN employees b
ON a.dept_id = b.dept_id
AND a.salary = b.salary
AND a.emp_id < b.emp_id;
```

83 **Write a query to show employees along with a 'salary bucket' label using CASE.**

Ans: CASE expressions work like IF-ELSE inside SELECT, returning custom labels based on conditions.

```
SELECT emp_name, salary,
CASE
WHEN salary >= 80000 THEN 'Premium'
WHEN salary >= 60000 THEN 'Senior'
WHEN salary >= 40000 THEN 'Mid'
ELSE 'Junior'
END AS salary_bucket
FROM employees
ORDER BY salary DESC;
```

84 **Find the department with the highest average salary.**

Ans: Use a subquery or LIMIT 1 with ORDER BY on a GROUP BY result.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
ORDER BY avg_sal DESC
LIMIT 1;
```

85 **Write a query to display a running count of employees hired each month.**

Ans: DATE_FORMAT extracts year-month; COUNT aggregates per group; result shows hiring trend by month.

```
SELECT DATE_FORMAT(hire_date, '%Y-%m') AS hire_month,
COUNT(*) AS new_hires
FROM employees
GROUP BY hire_month
ORDER BY hire_month ASC;
```

SECTION 17 — DCL (GRANT, REVOKE) & User Management

86 **Create a new MySQL user and grant SELECT and INSERT privileges.**

Ans: CREATE USER creates a login. GRANT gives specific privileges on specific objects.

```
-- Create user
CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'Str0ngPass!';

-- Grant SELECT and INSERT on one table
GRANT SELECT, INSERT ON company_db.employees TO 'analyst'@'localhost';

-- Apply changes immediately
FLUSH PRIVILEGES;
```

87 **Revoke INSERT privilege from a user and then drop the user.**

Ans: REVOKE removes specific privileges. DROP USER removes the account entirely.

```
-- Revoke INSERT only
REVOKE INSERT ON company_db.employees FROM 'analyst'@'localhost';

-- Remove all privileges and drop user
REVOKE ALL PRIVILEGES ON *.* FROM 'analyst'@'localhost';
DROP USER 'analyst'@'localhost';
```

SECTION 18 — Rapid-Fire Interview Questions with Queries

88 **What is the use of DISTINCT? Show an example.**

Ans: DISTINCT removes duplicate rows from the result set. Works on one or multiple columns.

```
SELECT DISTINCT dept_id FROM employees;
SELECT DISTINCT dept_id, job_title FROM employees;
```

89 **What does LIMIT with OFFSET do? Write a pagination query.**

Ans: LIMIT restricts rows returned. OFFSET skips rows. Together they paginate results.

```
-- Page 1 (rows 1-10)
SELECT * FROM employees ORDER BY emp_id LIMIT 10 OFFSET 0;

-- Page 2 (rows 11-20)
SELECT * FROM employees ORDER BY emp_id LIMIT 10 OFFSET 10;

-- Page 3 (rows 21-30)
SELECT * FROM employees ORDER BY emp_id LIMIT 10 OFFSET 20;
```

90 **How do you get the current date and time in MySQL?**

Ans: MySQL provides several functions for current date/time retrieval.

```
SELECT CURDATE(); -- 2024-06-15 (date only)
SELECT CURTIME(); -- 10:30:45 (time only)
SELECT NOW(); -- 2024-06-15 10:30:45
SELECT SYSDATE(); -- same as NOW()
SELECT UNIX_TIMESTAMP();
```

91 **How do you concatenate strings in MySQL?**

Ans: Use CONCAT() or CONCAT_WS() (with separator). The || operator is NOT supported in standard MySQL.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;

-- CONCAT_WS: first argument is the separator
SELECT CONCAT_WS(', ', last_name, first_name) AS name FROM employees;
```

92 How do you handle NULL in arithmetic and comparisons?

Ans: Any arithmetic with NULL returns NULL. Use IFNULL() or COALESCE() to substitute a default value.

```
SELECT salary + NULL; -- NULL
SELECT IFNULL(manager_id, 0) FROM employees; -- replace NULL with 0
SELECT COALESCE(phone, mobile, 'N/A') -- first non-NULL value
FROM employees;
```

93 What is AUTO_INCREMENT? How do you reset it?

Ans: AUTO_INCREMENT automatically generates a unique integer for each new row (typically the primary key). Reset with ALTER TABLE.

```
CREATE TABLE orders (order_id INT PRIMARY KEY AUTO_INCREMENT);

-- Insert without specifying order_id
INSERT INTO orders (customer_id) VALUES (5);

-- Reset counter to 1
ALTER TABLE orders AUTO_INCREMENT = 1;
```

94 How do you find the total number of rows in a table quickly?

Ans: COUNT(*) counts all rows. For a quick estimate on large InnoDB tables, use information_schema.

```
-- Exact count
SELECT COUNT(*) FROM employees;

-- Fast approximate count (large tables)
SELECT TABLE_ROWS FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'company_db'
AND TABLE_NAME = 'employees';
```

95 Write a query to show all tables in the current database.

Ans: SHOW TABLES lists all tables. information_schema.TABLES gives richer metadata.

```
SHOW TABLES;

-- With database name
SHOW TABLES FROM company_db;

-- With metadata
SELECT TABLE_NAME, TABLE_ROWS, TABLE_TYPE
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'company_db';
```

96 How do you describe the structure of a table?

Ans: DESC (short for DESCRIBE) shows column names, types, nullability, keys, defaults, and extras.

```
DESC employees;

-- Detailed version
SHOW FULL COLUMNS FROM employees;

-- Check the CREATE statement
SHOW CREATE TABLE employees;
```

97 What is the difference between UNION and UNION ALL?

Ans: UNION removes duplicate rows (performs implicit DISTINCT — slower). UNION ALL keeps all rows including duplicates (faster). Both require same number of columns with compatible data types.

```
-- UNION: distinct cities only
SELECT city FROM customers
UNION
SELECT city FROM suppliers;

-- UNION ALL: all cities including duplicates
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

98 Write a query to find employees who have the same salary as someone in a different department.

Ans: Self-join on salary, different dept_id to find cross-department salary matches.

```
SELECT DISTINCT a.emp_name, a.salary, a.dept_id
FROM employees a
JOIN employees b
ON a.salary = b.salary
AND a.dept_id != b.dept_id;
```

99 How do you use GROUP_CONCAT to list all employees per department in one row?

Ans: GROUP_CONCAT aggregates multiple row values into a single comma-separated string per group.

```
SELECT dept_id,
GROUP_CONCAT(emp_name ORDER BY emp_name SEPARATOR ', ') AS employees
FROM employees
GROUP BY dept_id;
```

100 **Write a query to find the manager who manages the most employees.**

Ans: Group by manager_id, count employees per manager, order DESC, limit to 1.

```
SELECT manager_id,
COUNT(*) AS team_size
FROM employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
ORDER BY team_size DESC
LIMIT 1;
```

100 MySQL Interview Questions Covered — Practice these queries on a live MySQL database to build real confidence for interviews.